

Smart Vision Assist — Project Architecture Report

Audited by: Technical Architecture Analysis
Date: April 28, 2026
Project Version: 1.0.0+1
Flutter SDK: ^3.11.5 | **Target Platform:** Android (API 36)

Table of Contents

- 1. Executive Summary
- 2. The Soul of the Project — What It's Really Doing
- 3. High-Level System Architecture
- 4. Complete Dependency Map
- 5. Core Engine Architecture
- 6. Feature Deep Dives
 - 6.1 Obstacle Detection — The Real-Time AI Pipeline
 - 6.2 GPS Navigation — From Voice to Route
 - 6.3 OCR — Dual-Engine Text Extraction
 - 6.4 Scene Description — AI Vision for the Blind
 - 6.5 Emergency System
 - 6.6 Settings & Preferences
- 7. The Decision Response Engine
- 8. Federated Learning Architecture
- 9. Voice & Audio Architecture
- 10. Bilingualism Architecture — How Arabic and English Coexist
- 11. Firebase Integration Map
- 12. Authentication & Settings Sync Flow
- 13. State Management Strategy
- 14. Android-Specific Implementation Details
- 15. Data Privacy Design
- 16. Performance Engineering
- 17. Error Handling & Resilience
- 18. The User Experience Flow — A Technical Walkthrough
- 19. Architecture Strengths & Technical Decisions

1. Executive Summary

Smart Vision Assist is an accessibility-first mobile application built for visually impaired users. It combines real-time on-device AI inference, cloud-based generative AI, GPS navigation, and voice interaction into a single, seamlessly bilingual (English/Arabic) experience. Every design decision in this codebase — from the decision engine's alert priority levels to the 10% federated learning prior strength — has been made to serve a user who cannot see the screen and relies entirely on audio feedback to navigate the physical world.

The application is not just a collection of features. It is a carefully orchestrated system where an obstacle 2 meters away will interrupt a navigation instruction, where a user can issue Arabic voice commands that are treated identically to their English counterparts, and where on-device TFLite models learn collectively through privacy-preserving federated learning without a single image ever leaving the device.

What makes this technically impressive:

- A dual-model object detector (YOLOv8n + EfficientDet Lite0) that auto-detects which model it loaded at runtime by inspecting tensor shapes
- A priority-based audio orchestration engine with cooldowns, interruption tokens, and navigation deferral
- A hand-rolled ZIP parser inside the TFLite model loader that extracts embedded label files from model metadata
- A federated learning system with a Python aggregation server and on-device 80-dim class-activation embeddings
- Full Arabic RTL support with normalized Arabic text search (diacritics stripped, $\delta \rightarrow \text{ö}$, etc.)
- ITU-R BT.601 YUV420→RGB conversion running in a Dart isolate to keep the UI at 60fps

2. The Soul of the Project

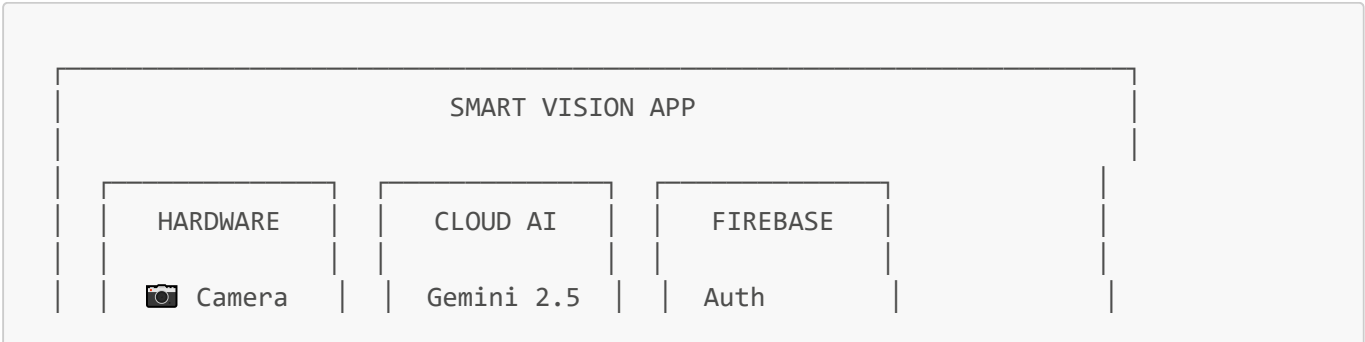
Before diving into code, it's important to understand **what this app is actually doing from a human perspective**, because every technical decision flows from it.

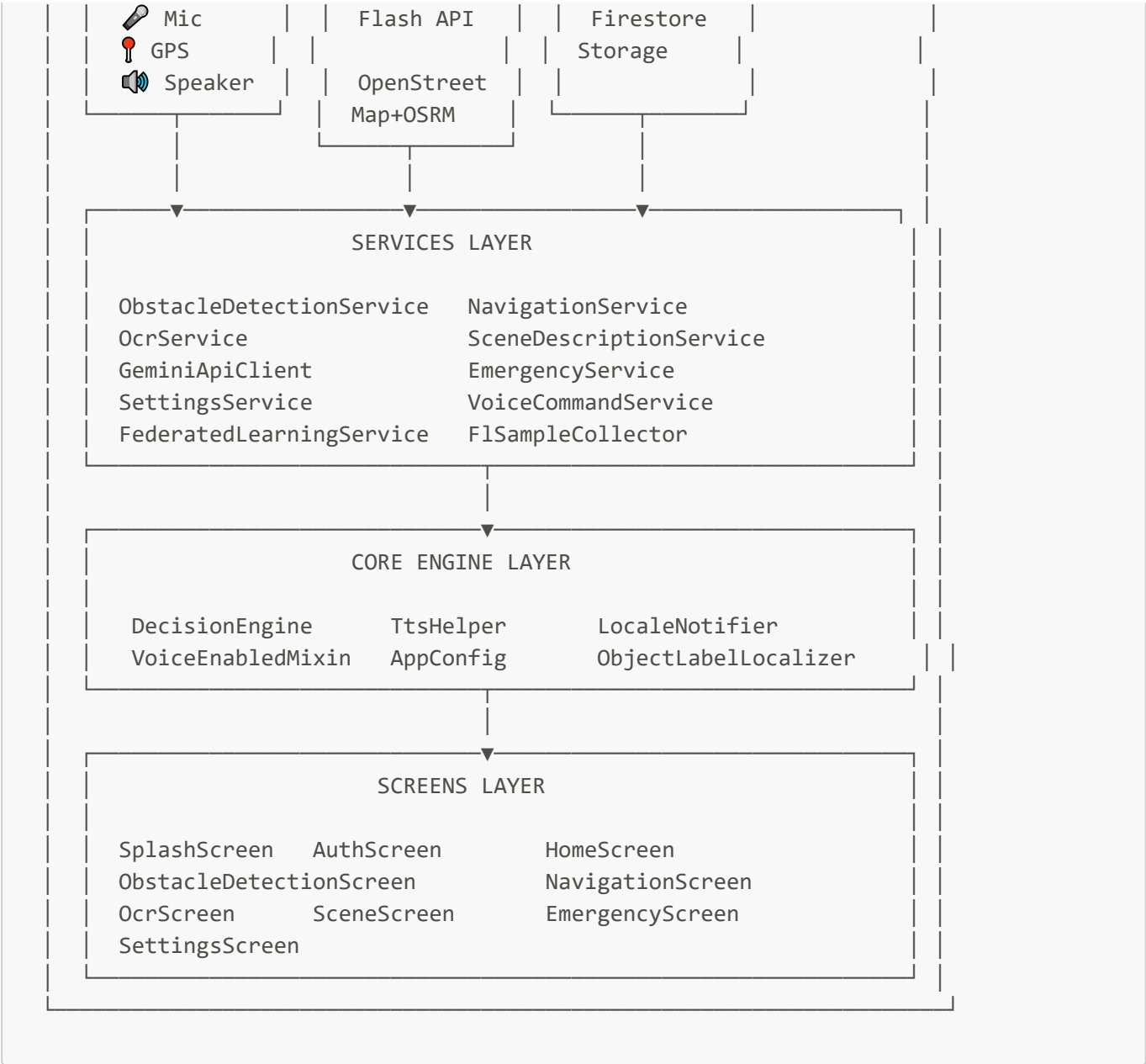
The user is visually impaired. They cannot look at the screen. They cannot read a map. They cannot see if there is a car coming. The entire app must communicate through sound. This single constraint shapes everything:

- Why is there a **DecisionEngine**? Because two audio alerts playing simultaneously is worse than silence — it creates confusion, not clarity.
- Why does obstacle detection run in an isolate? Because a dropped UI frame means a missed alert.
- Why does the OCR service use Gemini for Arabic but ML Kit for English? Because ML Kit simply doesn't support Arabic script recognition.
- Why is the federated learning prior strength exactly 0.10? Because a larger boost would cause false positives (phantom cars); a smaller boost would be meaningless for near-threshold detections.
- Why does the TTS helper have both **speakInterrupting()** and **speakPolite()**? Because "Car ahead, stop!" should cut off "In 50 meters, turn left", but "Bench detected" should not.

Every line of code is an answer to the question: "What does a blind user need right now?"

3. High-Level System Architecture





4. Complete Dependency Map

Every package in `pubspec.yaml` and the exact role it plays:

Package	Version	Role
<code>flutter_tts</code>	<code>^4.2.0</code>	Text-to-Speech — wraps Android TTS engine. The primary user output channel
<code>speech_to_text</code>	<code>^7.0.0</code>	Speech recognition — the primary user input channel
<code>camera</code>	<code>^0.11.0</code>	Live camera stream for obstacle detection (YUV420 frames)
<code>image_picker</code>	<code>^1.1.0</code>	Gallery/camera image selection for OCR and scene description
<code>image</code>	<code>^4.1.0</code>	Image manipulation: YUV→RGB conversion, resizing, JPEG encoding

Package	Version	Role
tflite_flutter	^0.12.1	TensorFlow Lite runtime — runs YOLO and EfficientDet models
google_mlkit_text_recognition	^0.13.0	On-device OCR for English/Latin text
firebase_core	^3.13.1	Firebase initialization
firebase_auth	^5.5.2	Email/password authentication
cloud_firestore	^5.6.6	Cloud storage for user settings and FL contribution metadata
firebase_storage	^12.4.4	Binary storage for FL global weights and model updates
geolocator	^12.0.0	GPS position for navigation
http	^1.2.1	HTTP client for Gemini, Nominatim, and OSRM APIs
url_launcher	^6.3.0	Opens SMS app and phone dialer for emergency features
intl	^0.20.2	Internationalization support
flutter_dotenv	^5.1.0	Loads .env file for API key management
shared_preferences	^2.3.2	Local key-value storage for user settings
path_provider	^2.1.5	Access to device file system directories
flutter_localizations	SDK	Flutter's built-in i18n support

5. Core Engine Architecture

The "core" directory contains the pieces that every screen and service depends on. Think of it as the operating system of the application.

AppConfig — The Environment Resolver

`lib/core/config/app_config.dart` is the single source of truth for runtime configuration. It implements a three-level priority chain for the Gemini API key:

Priority Order:

1. `--dart-define GEMINI_API_KEY` (CI/CD injection, highest security)

2. `.env` file via `flutter_dotenv` (local development)

3. Hardcoded demo fallback (demo mode, no real API calls)

It also provides the asset paths for both ML models:

- `assets/ml/efficientdet_lite0.tflite` — the default (EfficientDet SSD style)
- `assets/ml/Yolo-v8-Detection.tflite` — optional primary (YOLOv8n)

AppRoutes — Named Navigation

All screen transitions are managed through 8 named routes:

```
/           → SplashScreen
/auth       → AuthScreen
/welcome    → WelcomeScreen (future)
/home       → HomeScreen
/settings   → SettingsScreen
/emergency  → EmergencyScreen
/obstacle-detection → ObstacleDetectionScreen
/ocr        → OcrScreen
/scene-description → SceneDescriptionScreen
/navigation → NavigationScreen
```

LocaleNotifier — Runtime Language Switching

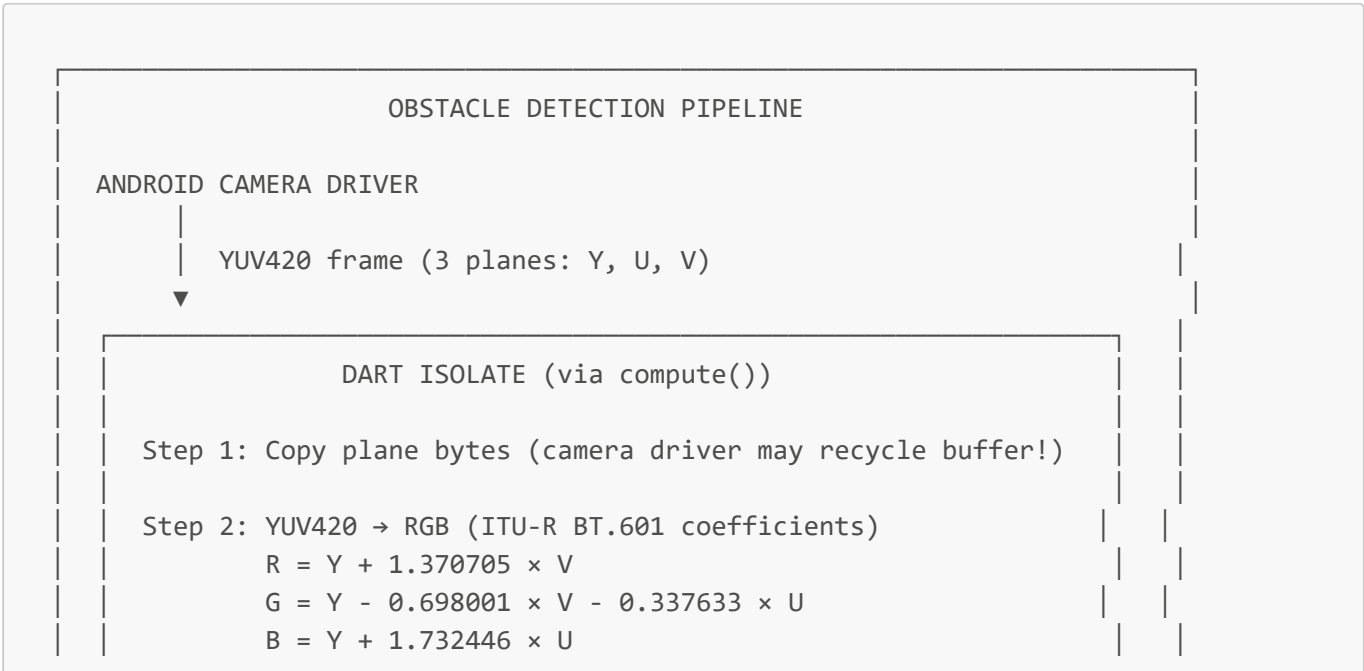
`lib/core/locale/locale_notifier.dart` is a `ValueNotifier<Locale>` singleton. When the user changes their language in Settings, this notifier fires, and the `MaterialApp.locale` rebuilds the entire widget tree with the new locale. This is how changing Arabic→English updates every piece of text in the app simultaneously without a restart.

6. Feature Deep Dives

6.1 Obstacle Detection — The Real-Time AI Pipeline

This is the most technically complex feature. It takes a live camera stream, runs it through a neural network in real-time, and speaks the results through a priority-based audio system. Here is exactly how it works, step by step.

The Full Detection Pipeline



Step 3: Rotate to correct orientation
 (Android sensors deliver sideways frames on portrait phones – YOLO trained on upright images!)
 90° → rotate +90 | 270° → rotate -90 | 180° → flip

Step 4: Resize to model input size
 YOLO: letterbox resize (preserve aspect, pad grey)
 SSD: direct square stretch
 Grey pad color: RGB(114, 114, 114)

Step 5: Pack into NHWC tensor
 Float32: normalize [0, 255] → [0.0, 1.0]
 Uint8: keep [0, 255] as-is

Step 6: (First frame only) Save debug JPEG to temp dir

Preprocessed tensor

TFLITE INTERPRETER (Main Thread)

Auto-detect model type by inspecting tensor shapes:

- 4+ outputs, one shape [B, N, 4] → SSD/EfficientDet
- 1 output, shape [B, C, A] or [B, A, C] → YOLO

YOLO Path:

Output shape: [1, 84, 8400] (84 channels × 8400 anchors)
 Channels 0-3: cx, cy, w, h (in 640-pixel space)
 Channels 4-83: class scores (80 COCO classes)
 Transposed variant: [1, 8400, 84] → auto-transpose

SSD Path:

Outputs: boxes [1,N,4], classes [1,N], scores [1,N],
 count [1]
 Class ID mapping: TF COCO 1-based (id+1 = COCO class)

Raw inference output

POST-PROCESSING

1. FL Prior Boost (YOLO only):
 $\text{boosted_score} = \text{raw_score} + \text{prior}[\text{class}] \times 0.10$
 (global weights from FederatedLearningService)
2. Confidence Threshold: discard if score < 0.35
3. Bounding Box Conversion (YOLO):
 $\text{left} = (\text{cx} - \text{w}/2) / \text{inputSize}$
 $\text{right} = (\text{cx} + \text{w}/2) / \text{inputSize}$
 $\text{top} = (\text{cy} - \text{h}/2) / \text{inputSize}$
 $\text{bottom} = (\text{cy} + \text{h}/2) / \text{inputSize}$

```

4. Greedy NMS (IoU threshold: 0.50):
   Sort by score descending → keep if IoU < 0.50 with all
   previously kept boxes

5. Direction from centerX:
   centerX < 0.4 → "Left" | 0.4-0.6 → "Centre" | > 0.6 → "Right"

6. Distance estimation (pinhole camera model):
   distance ≈ 1.8m / box_height_fraction
   (assumes average adult height = 1.8m)

7. Danger classification:
   dangerous = {person, car, motorcycle, bus, truck,
               bicycle, traffic light, stop sign}

8. FL Sample Collection (side channel):
   If 0.05 ≤ confidence ≤ 0.50 → save 80-dim embedding
   (fire-and-forget, never blocks inference)

Return: top 5 DetectedObstacle objects

```

DecisionEngine.onObstacleDetected()

TtsHelper.speak*()

 Android TTS Engine

Model Loading — The Self-Validating Loader

The `ObstacleDetectionService.initialize()` method implements a production-grade model loader with several impressive details:

- Version-suffixed cache:** The model is extracted from the APK assets to `{app_support_dir}/v6_{modelName}.tflite`. Bumping the `_modelVersion` constant to `v7` forces all users to re-extract on next launch.
- Atomic write:** The model is written to a `.tmp` file first, then renamed. This prevents a corrupted partial file from being loaded if the app crashes mid-write.
- Binary validation:** Before loading, the code checks:
 - File must be ≥ 1 MB (catches truncated downloads)
 - Must NOT start with ELF header `\x7fELF` (catches accidentally bundled `.so` native libs)
 - Must contain `TFL3` signature at bytes 4-7 (validates FlatBuffer format)
- Hand-rolled ZIP parser:** TFLite models can contain an embedded ZIP archive with metadata files including label lists. The code implements a complete ZIP central-directory parser in pure Dart

(`_readZipAssociatedFiles`, `_readZipFileData`) to extract these labels without any dependency. This is 100+ lines of careful byte-level parsing including CRC validation and zlib decompression.

5. **Label resolution fallback chain:** When looking up a class ID, the code tries:

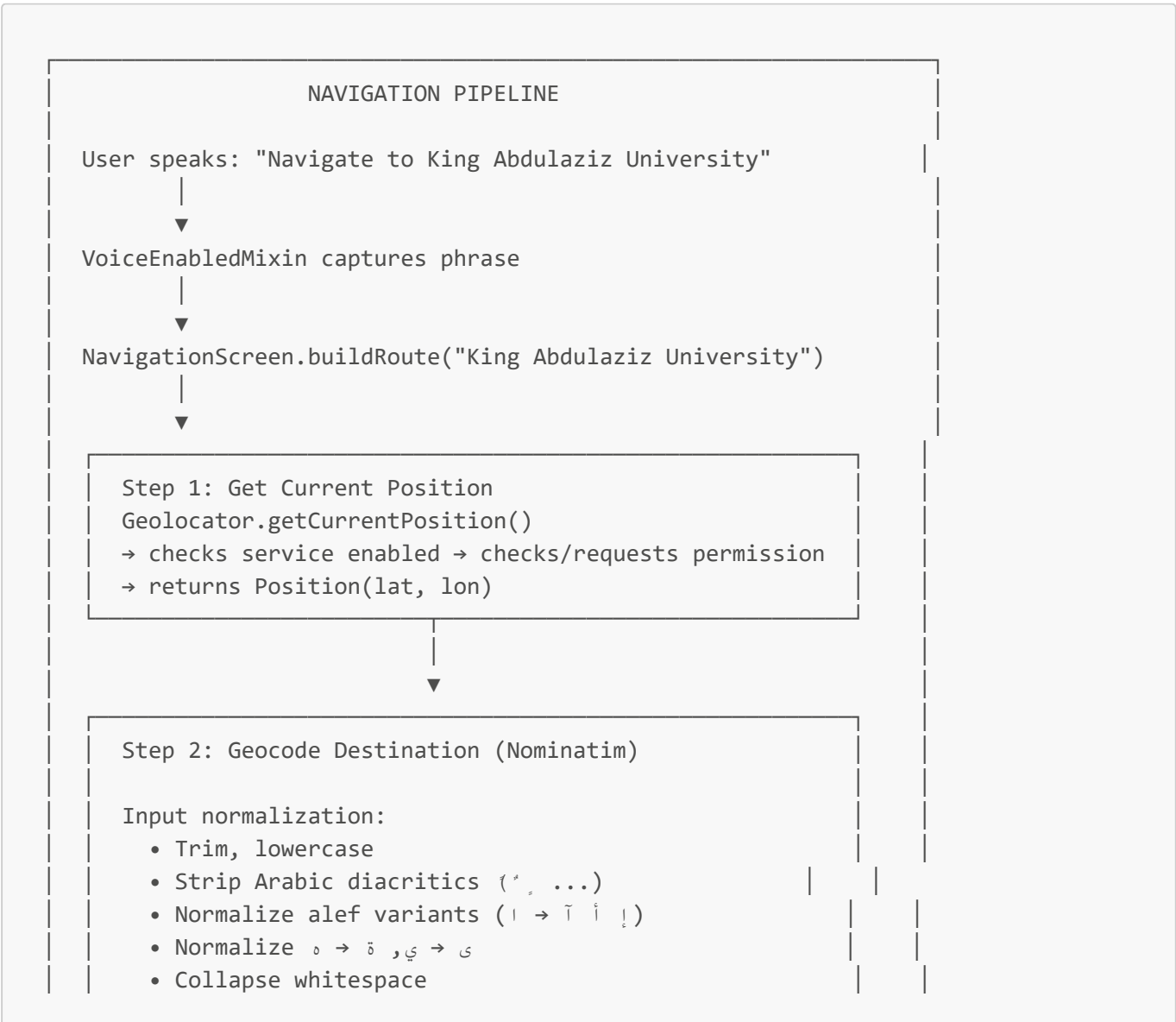
- Model metadata labels (highest priority)
- TF COCO 1-based mapping (accounts for reserved gap IDs like the missing class 12)
- Zero-based lookup
- Compact 80-class lookup

Demo Mode

On emulators or when the camera fails, `demoObstacles()` returns a hardcoded list that cycles through realistic scenarios. The `ObstacleDetectionScreen` uses a periodic timer to cycle through these, so developers can test the full audio pipeline without physical hardware.

6.2 GPS Navigation — From Voice to Route

The navigation feature chains four completely separate APIs together: Android GPS → OpenStreetMap Nominatim → OSRM Router → TTS. Here is the complete flow:



Build candidates list:

1. Original query (as-typed)
2. Place alias (e.g. "جامعة الملك عبد العزيز" → "King Abdulaziz University")
3. Normalized form
4. Reverse alias matches

For each candidate → GET nominatim.openstreetmap.org/search?q={candidate}&format=jsonv2&limit=1&accept-language={en|ar}

Returns: displayName, lat, lon



Step 3: Calculate Route (OSRM)

GET router.project-osrm.org/route/v1/walking/{currentLon},{currentLat};{destLon},{destLat}?overview=false&steps=true&alternatives=false

Walking profile (not driving!) – correct for visually impaired pedestrian use
 overview=false reduces payload size
 alternatives=false (fastest only)



Step 4: Parse Maneuver Instructions

For each step in route.legs.steps:
 maneuver.type → localized instruction

Supported maneuver types:

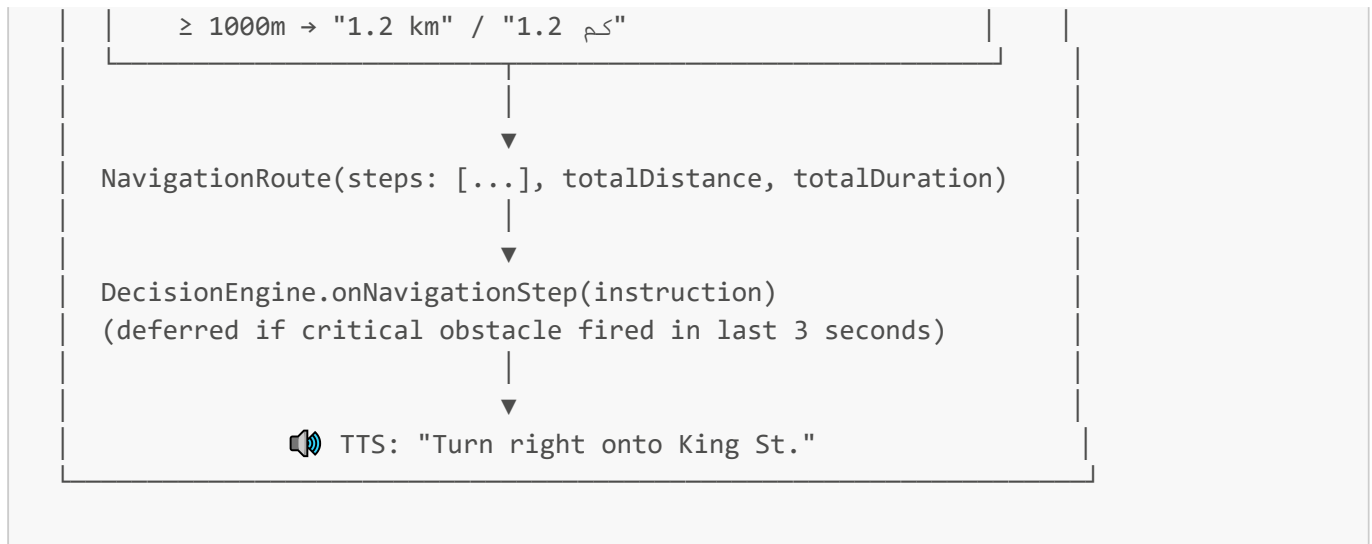
depart → "Start walking on {road}"
 turn → "Turn left/right onto {road}"
 continue → "Continue straight on {road}"
 fork → "Keep left/right at the fork"
 roundabout → "Enter the roundabout"
 arrive → "You have arrived at your destination"
 end of road → "At the end of the road, turn..."
 merge → "Merge left/right"

Modifier localization (Arabic):

left → يساراً | right → يميناً
 slight left → قليلاً إلى اليسار
 sharp right → بحدة إلى اليمين
 u-turn → للخلف

Distance labels:

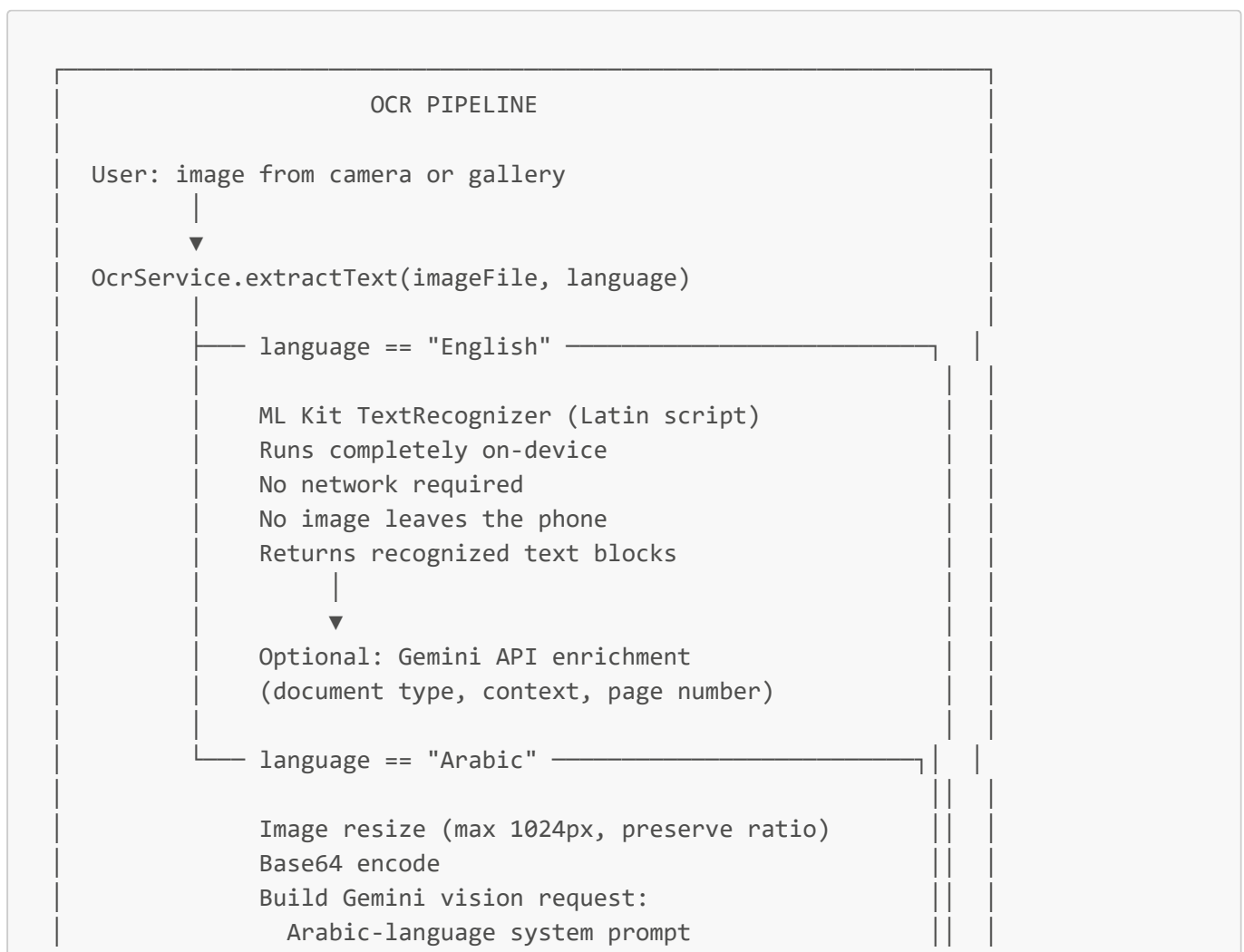
< 1000m → "450 m" / "450 م"

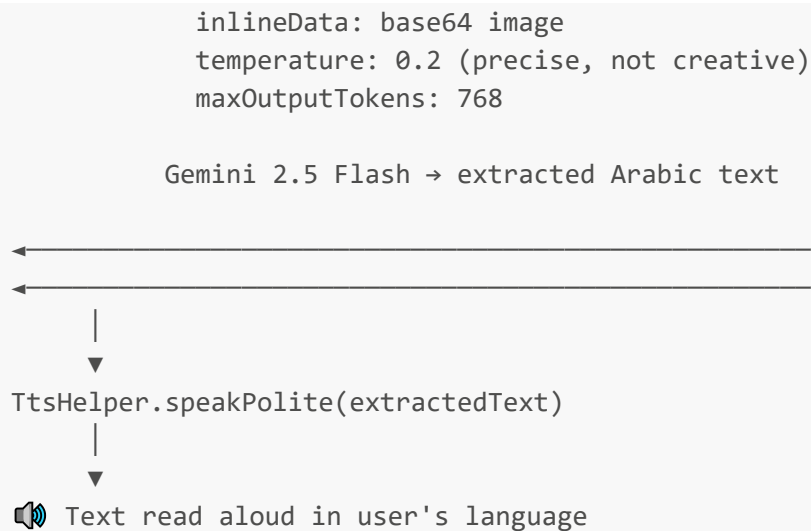


The place alias system is a notable UX feature. The map `_placeAliases` in `navigation_service.dart` translates known local landmarks — "رد سي مول" → "Red Sea Mall", "جامعة جدة" → "University of Jeddah" — before the geocoding request. This means users can say local nicknames and the system still finds the right place.

6.3 OCR — Dual-Engine Text Extraction

The OCR feature uses completely different engines depending on the language, because the problem is fundamentally different for Arabic vs. English text.

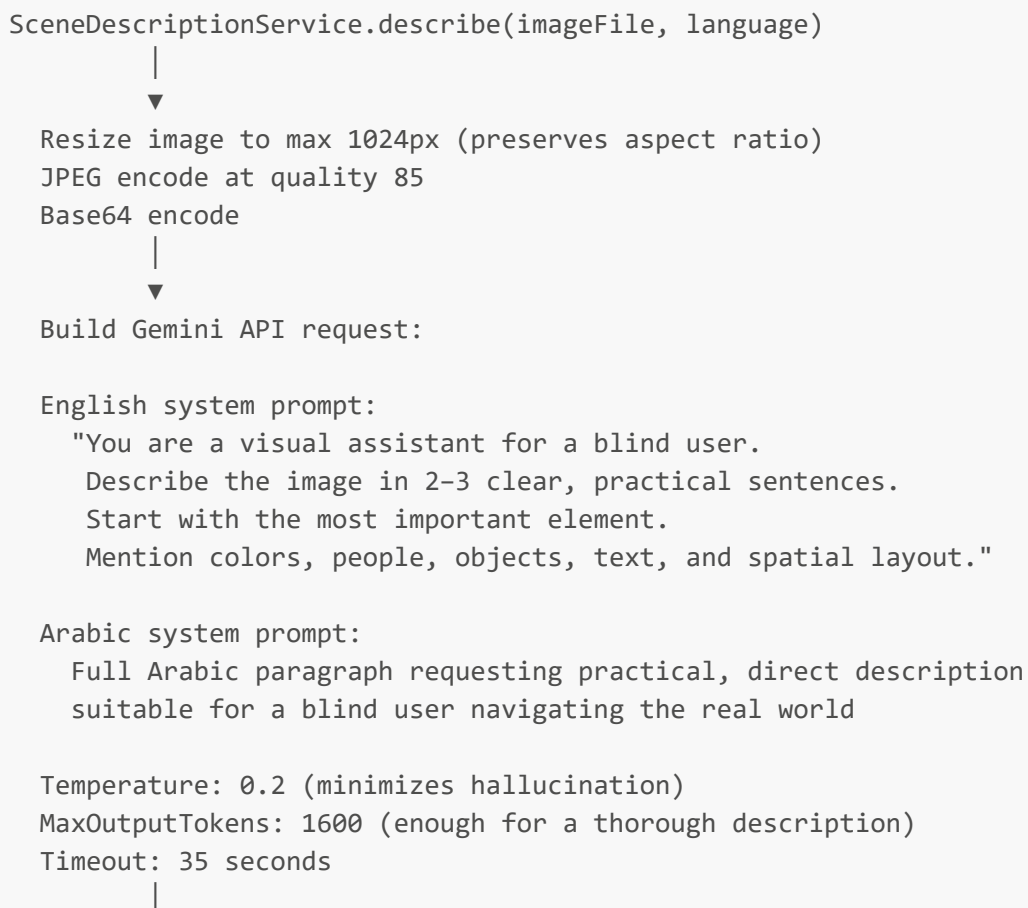


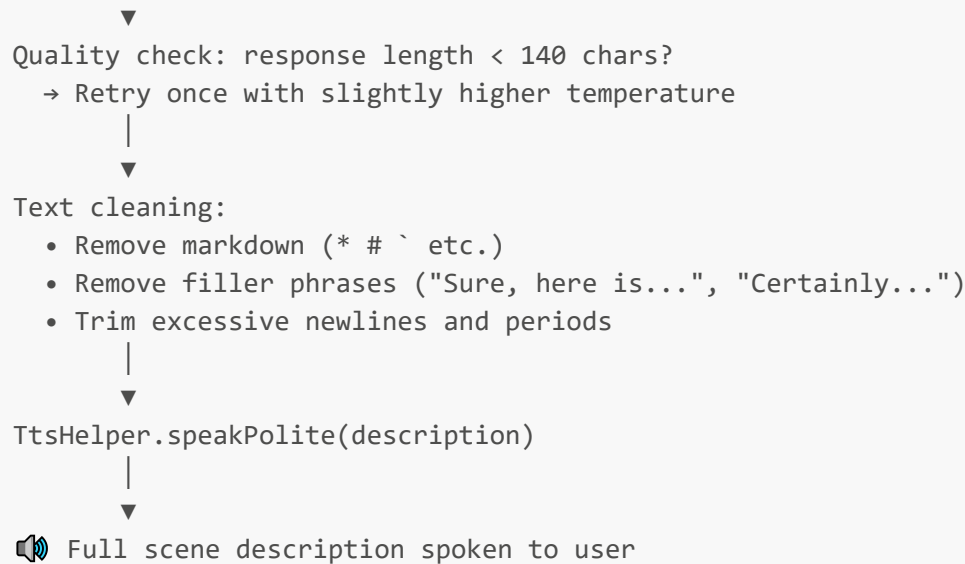


Why Gemini for Arabic? Google ML Kit's `TextRecognizer` supports Latin script extremely well, but Arabic is a cursive, right-to-left script with ligatures that make character-level recognition much harder. The Gemini 2.5 Flash model, trained on massive multilingual datasets including Arabic text, delivers significantly better results on Arabic documents than available on-device OCR engines.

6.4 Scene Description — AI Vision for the Blind

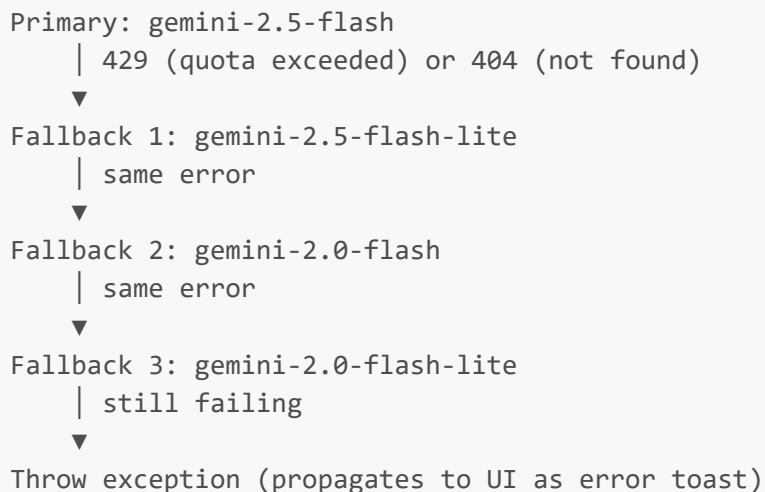
This feature is the most direct expression of the app's core purpose. The user points the camera at something and hears a full description of what they're looking at.





Gemini Model Fallback Chain

The `GeminiApiClient` implements smart retry with model degradation:



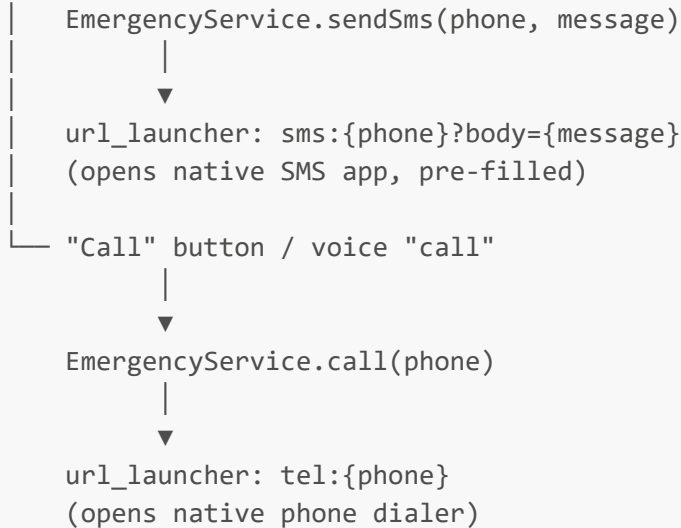
Non-retryable errors (5xx server errors, malformed requests) fail immediately without trying fallbacks.

6.5 Emergency System

The emergency feature is intentionally simple by design — in a crisis, complexity kills. A visually impaired user needs to press one button (or say one phrase) and know help is on the way.

Architecture:





The emergency contact (name + phone) is stored in `SettingsService` (SharedPreferences) and synced to Firestore. The SMS template message is localized — it reads differently in Arabic vs. English.

Importantly, the app uses `url_launcher` rather than directly sending SMS programmatically. This is both a permission and a UX decision: the user still confirms before the SMS is sent, which is actually desirable in a system that responds to voice commands (preventing accidental sends).

6.6 Settings & Preferences

Settings are stored in three places simultaneously:



Settings stored:

- `language`: "English" or "Arabic"
- `voiceSpeed`: 0.3 (slow) / 0.45 (normal) / 0.65 (fast)
- `alertVolume`: 0.0–1.0
- `contactName`, `contactPhone`: emergency contact

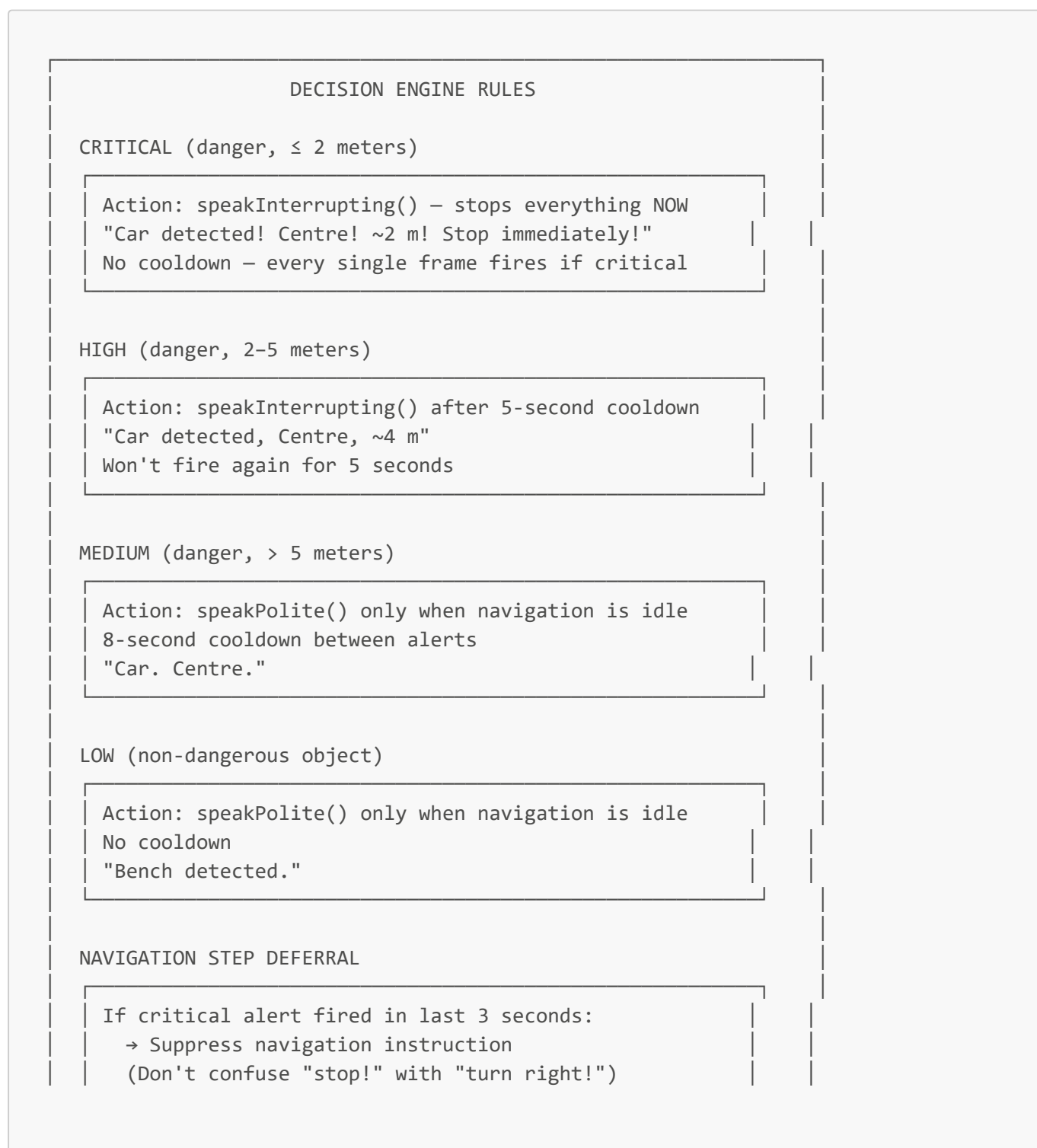
- `federatedLearningEnabled`: boolean

7. The Decision Response Engine

The `DecisionEngine` is the piece of code that makes the difference between a helpful app and a chaotic one. Without it, the TTS would try to say "Car detected, 3 meters, centre" at the same time as "In 50 meters, turn right" — creating an unintelligible audio soup.

The engine is a singleton that mediates all audio output from two competing sources: obstacle detection and navigation instructions.

Priority Levels





State Coordination

The engine tracks two booleans: `_navigationActive` and `_detectionActive`. Screens tell the engine their state:

```
// ObstacleDetectionScreen:
DecisionEngine.instance.setDetectionActive(true);

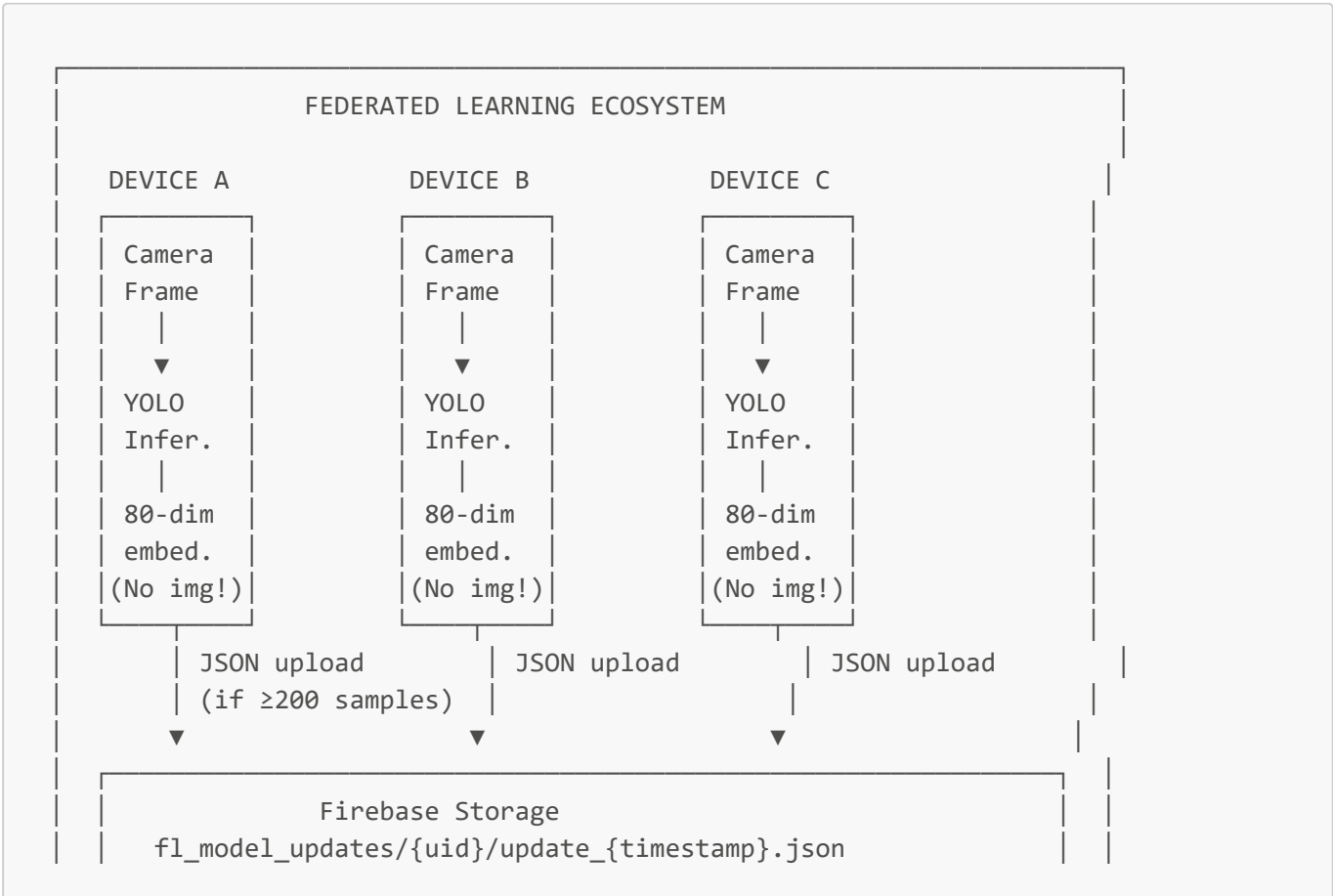
// NavigationScreen:
DecisionEngine.instance.setNavigationActive(true);
```

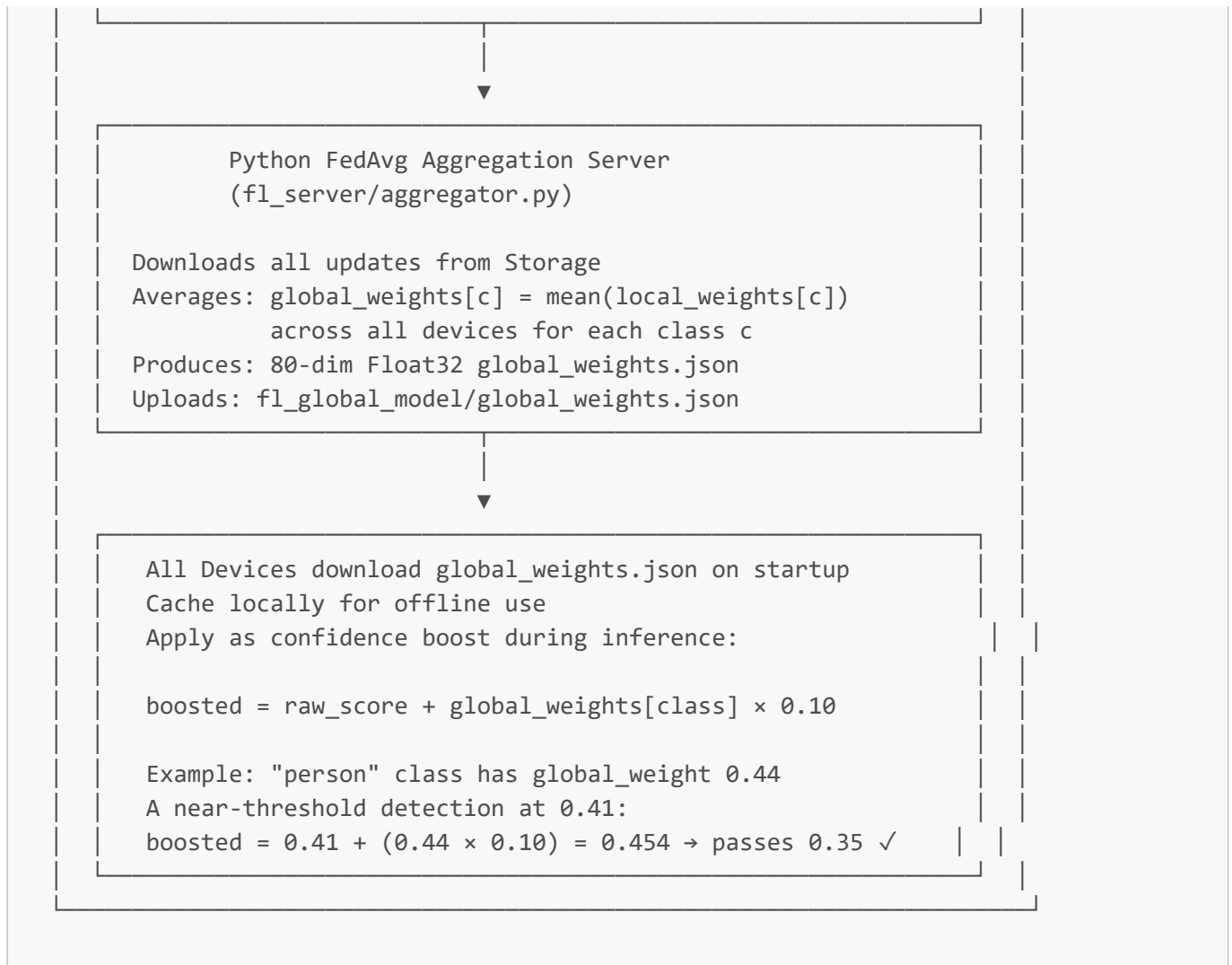
When both features run simultaneously (a common real-world scenario — navigating while watching for obstacles), the engine coordinates between them using these states plus the `_lastDangerAlert` timestamp.

8. Federated Learning Architecture

This is the most architecturally innovative feature. The app participates in a distributed machine learning system where all devices collectively improve the obstacle detection model's confidence scores, while no images ever leave any device.

The Big Picture





What Gets Uploaded (and What Doesn't)

Uploaded:

```

{
  "schema_version": 1,
  "update_type": "class_activation_average",
  "reason": "threshold_reached",
  "created_at": "2026-04-28T10:30:00Z",
  "sample_count": 247,
  "confidence_mean": 0.312,
  "local_weights": [0.21, 0.44, 0.18, ...], // 80 floats
  "class_counts": [12, 33, 8, ...], // 80 ints
  "privacy": { "images_uploaded": false, "on_device_only": true }
}
  
```

Never uploaded: Any camera image. Any raw frame. Any pixel data.

The 80-Dim Embedding

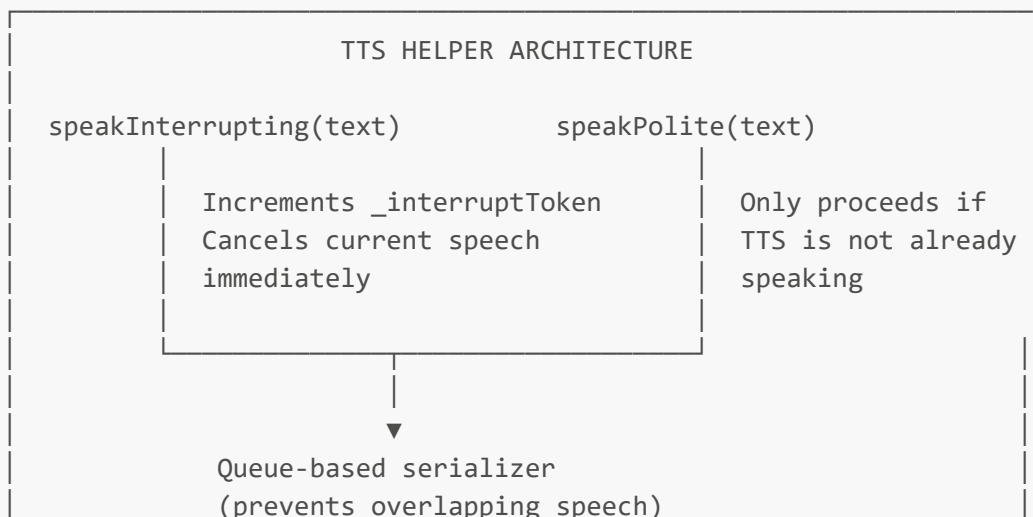
The class-activation embedding is built as follows: for every anchor in the YOLO output (all 8400 of them), find the maximum class score for each of the 80 COCO classes across all anchors. The result is a single 80-

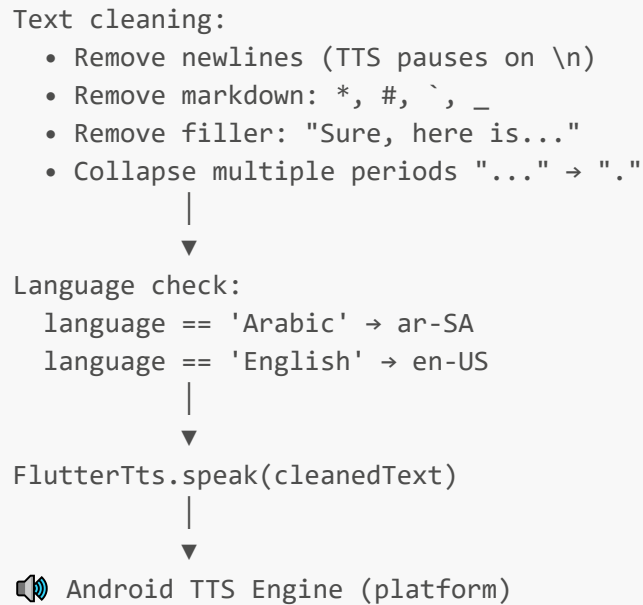

```
embedding[c] = max(score[c][anchor]) for anchor in [0, 8400)
```

FL Sample Collection Window

Below 0.05: too noisy to learn from
Above 0.50: model already confident, no improvement needed
0.05-0.50: "uncertain" zone where global knowledge helps most

- **Threshold reached:** 200 samples collected (`kFlRoundSampleThreshold`)
- **Manual sync:** User presses "Sync Now" in Settings
- **Manual build:** User presses "Build Update" in Settings





The Interrupt Token System

When `speakInterrupting()` is called, it increments a counter (`_interruptToken`). Before each async TTS call completes, the code checks whether the token is still the same. If a newer interrupt was issued, the older call silently drops its result. This prevents a race condition where:

1. Critical alert starts speaking
2. Another critical alert fires
3. The first alert's completion callback tries to resume speech

Without the token, both alerts would play sequentially. With the token, only the most recent one speaks.

VoiceEnabledMixin — Voice Command Composition

Every screen that needs voice commands uses this mixin. It standardizes voice command behavior across the entire app:

```

// Example: ObstacleDetectionScreen uses the mixin
class _ObstacleDetectionScreenState extends State<ObstacleDetectionScreen>
  with VoiceEnabledMixin {

  @override
  Map<List<String>, VoidCallback> get voiceCommandMap => {
    ['start', 'detect', 'begin']: _startDetection,
    ['stop', 'pause', 'end']: _stopDetection,
    ['repeat', 'again', 'last alert']: _repeatLastAlert,
  };
}
  
```

The mixin handles:

- Auto-starting the microphone 900ms after the screen loads (configurable delay)
- Matching any recognized phrase against the command map
- Announcing "Listening..." or "I didn't understand" via TTS
- Auto-restarting recognition after unknown commands
- Stopping the microphone on `dispose()`

ObjectLabelLocalizer — Bilingual Alert Construction

`ObjectLabelLocalizer` contains a hand-crafted 88-entry dictionary mapping COCO class names to Arabic translations, plus templates for building full alert sentences:

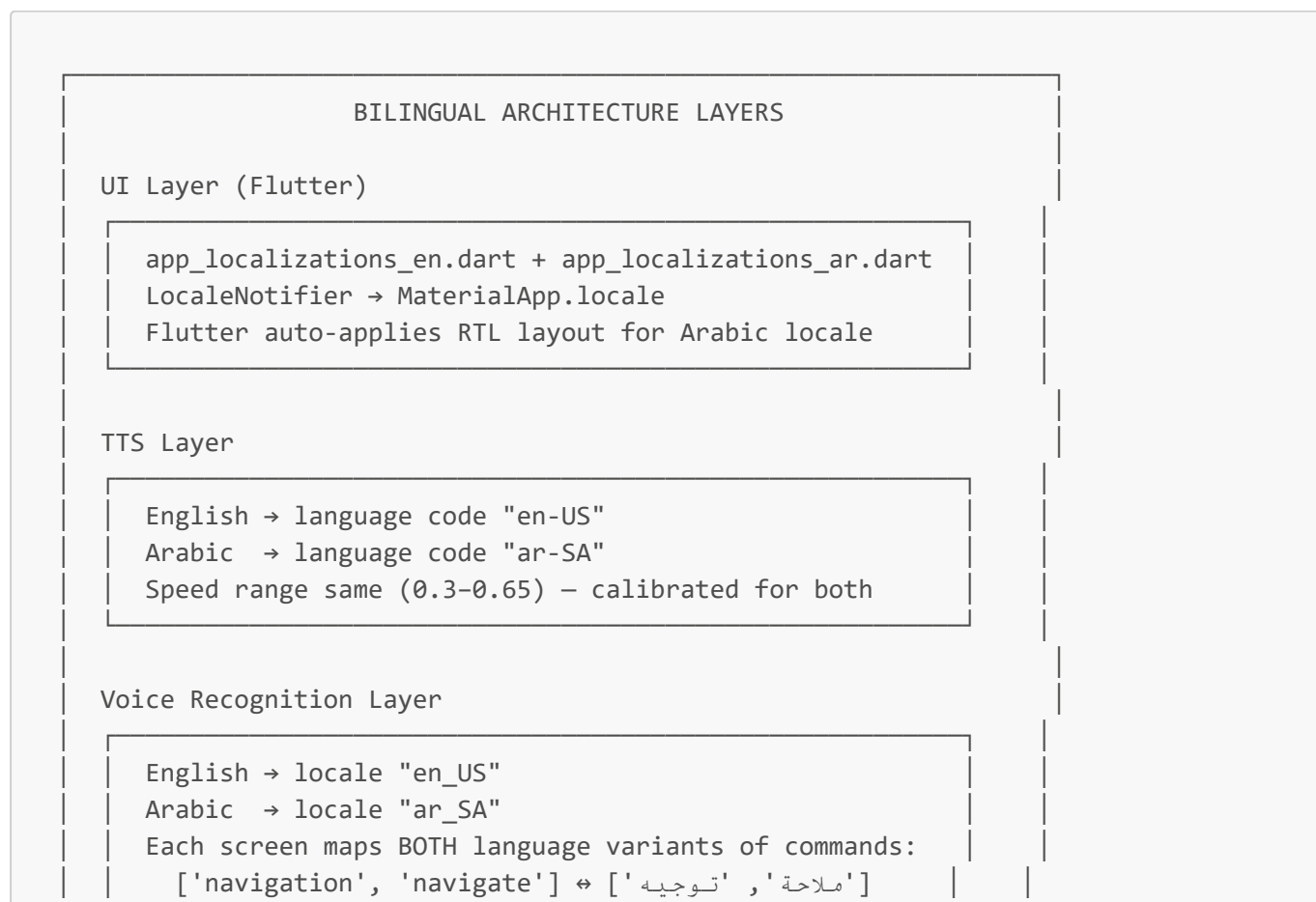
```
English: "Warning! Car detected. Centre. ~3 m. Stop immediately."  
Arabic:  "تحذير! تم اكتشاف سيارة. الوسط. ~3 م. توقف فوراً"
```

The 88 translations cover every dangerous and common COCO class, including Arabic-appropriate terms (not just transliterations).

10. Bilingualism Architecture

Supporting Arabic is not just about translating strings. Arabic is right-to-left, has a different number system context, different voice synthesis requirements, different OCR engines, and different text normalization rules. Here's how every layer handles bilingualism.

Layer-by-Layer Bilingual Support



OCR Layer

English → ML Kit TextRecognizer (Latin script, local)
Arabic → Gemini Vision API (cloud)

Object Labels Layer

ObjectLabelLocalizer: 88 COCO classes → Arabic names
Direction terms: Left/Right/Centre → يسار/يمين/الوسط
Tip terms: "Stop immediately" → "توقف فوراً"

Navigation Layer

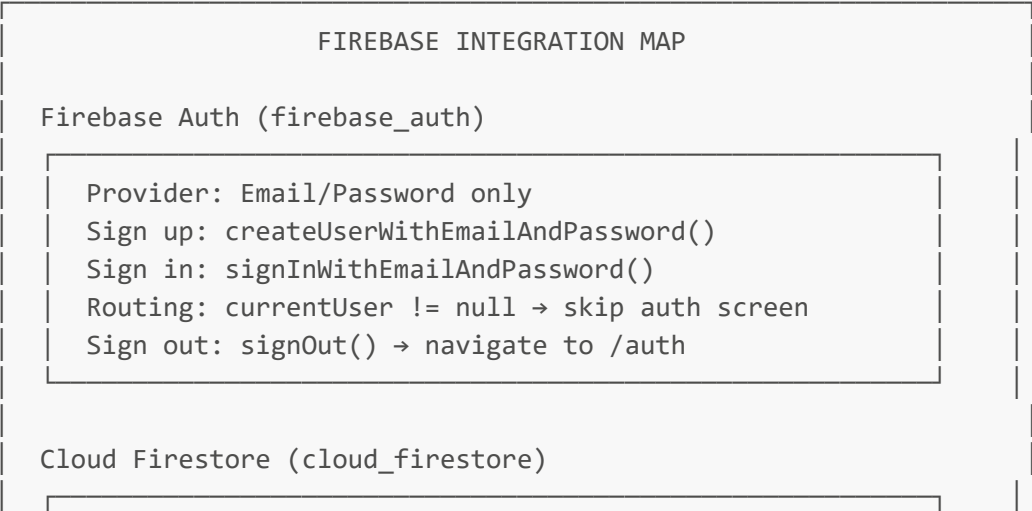
Nominatim: accept-language header (en|ar)
OSRM instructions → localized maneuver templates
8 modifier types fully translated

Search Normalization (Arabic-specific)

Strip diacritics: "مكة" → removed
Alef unification: ا → آ إ إ
Ya normalization: ي → ي
Ta marbuta: ه → ة
This ensures "جامعة جدة" matches "جامعة جدة"

11. Firebase Integration Map

The app uses Firebase in three distinct ways:



Document structure:

```

users/
  {uid}/
    settings/
      prefs → {
        language: "Arabic",
        voiceSpeed: 0.45,
        alertVolume: 0.7,
        contactName: "Ahmed",
        contactPhone: "+966...",
        federatedLearningEnabled: true
      }

fl_contributions/
  {uid}/
    updates/
      {timestamp} → {
        sample_count: 247,
        confidence_mean: 0.312,
        created_at: ISO8601,
        reason: "threshold_reached",
        privacy: {images_uploaded: false}
      }

```

Firebase Storage (firebase_storage)

Paths:

```

fl_global_model/
  global_weights.json      ← aggregated weights
                           (written by Python server)

fl_model_updates/
  {uid}/
    update_{timestamp}.json ← local FL updates
                           (written by app)

```

All Firestore operations are wrapped in try/catch with silent no-ops when offline. The app never requires connectivity to function — Firebase is additive, not required.

12. Authentication & Settings Sync Flow

```

APP LAUNCH
  |
  ▼
flutter_dotenv.load('.env')      // Load API keys

```



loadFromFirestore() hydrates local SharedPreferences from the cloud document, then applies the settings to TTS and locale. This means a user switching devices gets their preferred language and speed immediately after signing in.

13. State Management Strategy

The app deliberately avoids heavy state management frameworks like Riverpod, Bloc, or Provider. Here's the intentional pattern:

Mechanism	Used For	Why
ValueNotifier<Locale>	Language changes (LocaleNotifier)	Drives MaterialApp.locale rebuild
StreamController	FL sample count updates	Settings screen shows live sample count
StatefulWidget.setState()	All screen-local state	Simple, no overhead for local UI
Singleton instances	TtsHelper, DecisionEngine, FederatedLearningService,	Single authoritative instance across the app

Mechanism	Used For	Why
	FlSampleCollector, SettingsService	
Mixin (VoiceEnabledMixin)	Voice command behavior	Composable, avoids code duplication

The **singleton pattern** is used extensively and intentionally. The **DecisionEngine** must be a singleton because it coordinates state between two different screens (obstacle detection + navigation) that may be open simultaneously in theory, but more critically, because it holds the **_lastDangerAlert** timestamp that both screens need to share.

14. Android-Specific Implementation Details

Permissions Declared

```
<!-- Required for all API calls and service communication -->
<uses-permission android:name="android.permission.INTERNET"/>

<!-- Obstacle detection (live camera) -->
<uses-permission android:name="android.permission.CAMERA"/>
<uses-feature android:name="android.hardware.camera" android:required="false"/>

<!-- Voice input -->
<uses-permission android:name="android.permission.RECORD_AUDIO"/>

<!-- GPS navigation -->
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>

<!-- Image picker (gallery) - Android version-conditional -->
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>      <!--
API 33+ -->
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>    <!--
API ≤32 -->
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>  <!--
API ≤28 -->
```

Build Configuration

- **compileSdk / targetSdk: 36** (latest stable)
- **minSdk: flutter default** (typically API 21, Android 5.0)
- **Java 17 source/target compatibility**
- **Kotlin 2.2.20**
- **noCompress += ["tflite", "lite"]** — This is critical. Without this, Android's APK packager would compress the TFLite model files. But TFLite needs to memory-map (mmap) the model directly from the APK, which only works on uncompressed assets. This single line prevents model loading failures.

ProGuard Rules

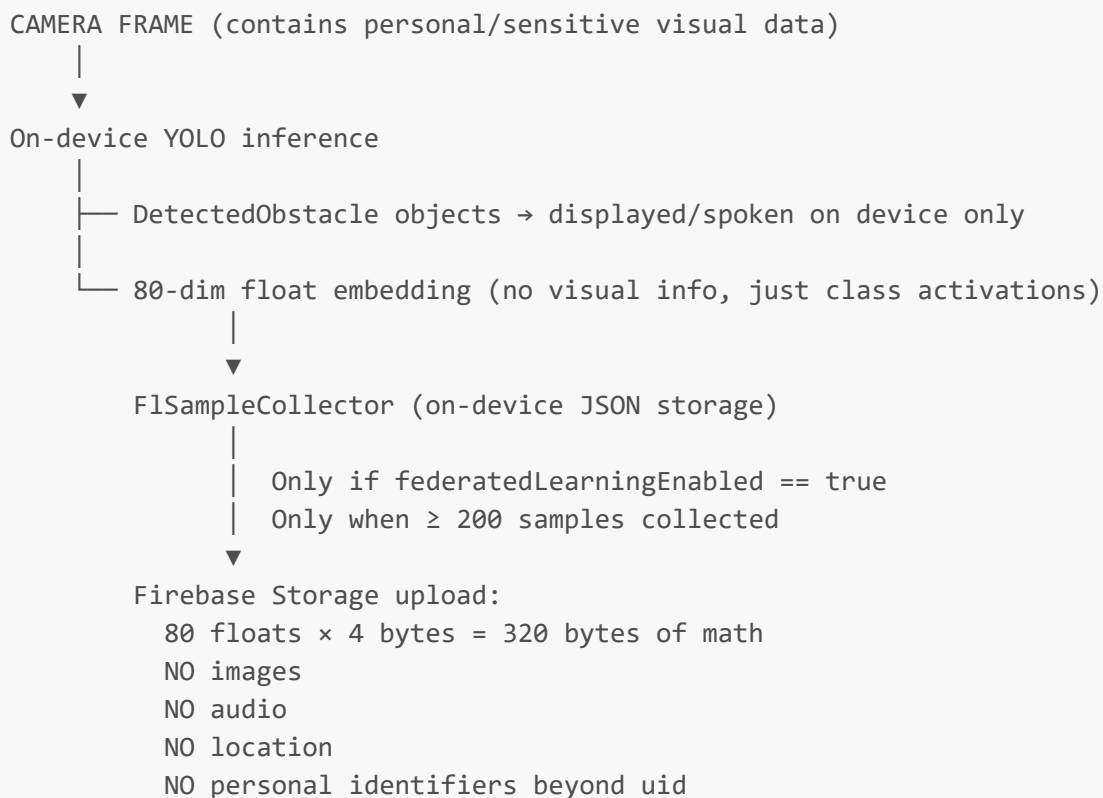
```
-keep class com.google.mlkit.**          # ML Kit text recognition
-keep class org.tensorflow.**            # TFLite GPU delegate
-keep class com.google.android.gms.**    # Google Play Services
```

Native Channel for Debug Images

A custom `MethodChannel('smart_vision_app/debug_images')` is used to save preprocessed detector input images to the device gallery during development. This is how developers verify that the YUV→RGB conversion, rotation, and letterbox resizing are working correctly on a real device — they can open the gallery and see exactly what the model saw.

15. Data Privacy Design

The privacy architecture follows a clear principle: **process locally, share only mathematical abstractions.**



This architecture achieves what pure differential privacy aspires to: even if Firebase were compromised, an attacker would learn that "device {uid} tended to detect class 'person' with 44% confidence" — nothing more. The original images that produced those statistics are gone.

16. Performance Engineering

Several non-obvious performance decisions are worth highlighting:

Isolate-Based Preprocessing


```
final preprocessed = await compute(_preprocessFrame, _FrameArgs(...));
```

The `_preprocessFrame` function (YUV→RGB conversion, rotation, resize) runs in a background Dart isolate via `compute()`. On a frame resolution of, say, 640×480, this is ~921,600 pixel conversions using the BT.601 matrix — enough work to drop frames if run on the UI thread. The isolate keeps the UI at 60fps.

Why `compute()` and not a raw `Isolate`? `compute()` is Flutter's convenience wrapper that serializes the function + arguments, spawns an isolate, runs the function, and deserializes the result. It's simpler for one-shot work. The limitation (interpreters can't be shared across isolates) is why TFLite inference runs back on the main thread after the isolate returns.

Plane Buffer Copying

```
yBytes: Uint8List.fromList(frame.planes[0].bytes),
```

This copy-before-send looks wasteful but is necessary. Android camera drivers may recycle the buffer before the isolate processes it. Without copying, the isolate could read garbage data or crash.

Model Cache with Version Suffix

The model is cached at `{app_support_dir}/v6_{modelName}.tflite`. When the model changes, bumping `_modelVersion` to `v7` ensures all users get the new model on next launch. Without versioning, users would keep using the old cached model.

Frame Skipping

The detection screen uses a boolean flag `_isProcessingFrame`. When true, incoming camera frames are dropped rather than queued. This prevents a backlog of frames from building up when inference takes longer than the camera frame rate.

Single-Threaded TFLite

```
final options = InterpreterOptions()..threads = 1;
```

Counter-intuitively, `threads = 1` is faster on budget Android phones like the Samsung A06 (the development target mentioned in code comments). With multiple threads, TFLite uses XNNPACK or other backends that have significant thread synchronization overhead for small models on slow CPUs. A single thread avoids this.

17. Error Handling & Resilience

The app uses a layered approach to failures:

Failure Type	How It's Handled
No internet	All operations fail gracefully; cached data (FL weights, settings) used
Gemini API 429	Automatic model downgrade through fallback chain
Gemini timeout (35s)	Exception propagates to UI as error toast
TFLite model corrupt	Detected at startup via binary validation; clear error thrown
TFLite model too small	Detected at startup (< 1 MB); triggers re-extraction
GPS unavailable	NavigationException with localized message spoken via TTS
Nominatim not found	Retries with all alias candidates before throwing
Camera unavailable	Falls back to demo mode with hardcoded obstacles
Firebase offline	All Firestore calls wrapped in try/catch; silent no-ops
FL upload failure	Silent no-op; samples remain on device for next attempt
Speech not recognized	VoiceEnabledMixin speaks "I didn't understand" and restarts listening

The guiding principle: **the user should never hear silence when something goes wrong**. Every error path either shows a fallback result or speaks an error message.

18. The User Experience Flow — A Technical Walkthrough

Here is the complete journey of a first-time user, from app launch to using every feature:

1. APP LAUNCH

SplashScreen renders for 2 seconds
Firebase checks auth state → no user found
Routes to AuthScreen

2. AUTHENTICATION

AuthScreen renders with email + password fields
TTS speaks: "Welcome to Smart Vision Assist. Please sign in."
User speaks email and types password
Taps "Sign Up" → Firebase creates account
SettingsService.syncToFirestore() creates cloud settings document
Routes to HomeScreen

3. HOME SCREEN

6 feature cards displayed
TTS speaks welcome message
Microphone starts automatically after 900ms
User can say "Navigation" / "ملاحة" to go to navigation
Or tap any card

4. OBSTACLE DETECTION

ObstacleDetectionScreen opens
Camera initializes → CameraController starts stream

```
ObstacleDetectionService.initialize() extracts and validates model
DecisionEngine.setDetectionActive(true)
Every frame: isolate preprocesses → TFLite infers → alerts spoken
User hears: "Warning! Car detected. Centre. ~2 m. Stop immediately!"
On pause/background: camera stream stops, model stays loaded
```

5. NAVIGATION

```
NavigationScreen opens
User speaks or types destination: "King Abdulaziz University"
GPS acquired → Nominatim geocodes → OSRM routes
DecisionEngine.setNavigationActive(true)
Steps displayed and spoken one by one
If obstacle detected simultaneously: critical alerts override nav steps
```

6. OCR

```
OcrScreen opens
User takes photo of a document/sign
English → ML Kit → text spoken
Arabic → Gemini Vision → text spoken
```

7. SCENE DESCRIPTION

```
SceneDescriptionScreen opens
User takes photo
Gemini analyzes → 2-3 sentence description spoken
"A busy street scene. A pedestrian crossing with traffic lights
on red. Buildings on both sides in beige and white colors."
```

8. EMERGENCY

```
User says "Send alert" or taps button
SMS app opens with pre-filled message to contact
User says "Call" → phone dialer opens
```

9. SETTINGS

```
Language switched → entire UI rebuilds in Arabic + RTL layout
Speed changed → TTS immediately reflects new speed
FL toggle enabled → sample collection begins on next detection session
```

19. Architecture Strengths & Technical Decisions

The Most Technically Impressive Details

1. Hand-Rolled ZIP Parser for Model Metadata The model loader contains a complete ZIP file parser in pure Dart, implemented to extract label files embedded inside TFLite model archives. This is ~120 lines of careful binary parsing (central directory, local file headers, CRC32, DEFLATE via ZLib) written to avoid any external dependency. It tries multiple label file formats (`.txt` lists, `.json` with `names` keys or `names` dicts) and multiple naming conventions (`label.txt`, `classes.txt`, etc.).

2. Runtime Model Architecture Detection The service doesn't need to be told which model is loaded. It inspects the output tensor shapes at runtime and decides:

- `[B, N, 4]` shaped output among multiple outputs → SSD/EfficientDet

- Single output [B, C, A] or [B, A, C] → YOLO
- This means swapping the model file is enough to switch detectors. No code changes required.

3. BT.601 YUV→RGB in an Isolate The YUV420 to RGB conversion uses the correct ITU-R BT.601 broadcast television coefficients (not approximations), runs in a background Dart isolate to avoid UI jank, and copies plane bytes before sending to prevent camera buffer recycling issues. This is production-grade camera preprocessing.

4. Letterbox Resize for YOLO YOLO models are trained with letterbox padding (grey borders), not stretched square crops. Stretching makes objects look unnatural and significantly reduces confidence scores. The code uses grey (114, 114, 114) padding — the same grey used in YOLOv5/YOLOv8 official training — and correctly centers the content.

5. Arabic Text Search Normalization The destination search normalizes Arabic text before geocoding. It strips diacritics (ﻉَ), unifies alef variants (ا → آ إ إ), and normalizes ta marbuta (ة → ة). This means a user who writes "جامعه جده" (informal, unvoweled) matches "جامعة جدة" (formal) in the Nominatim API. Arabic users never use diacritics in casual typing.

6. The Decision Engine's 3-Second Danger Window The 3-second deferral window for navigation instructions after critical alerts is carefully calibrated. An adult needs roughly 1-2 seconds to process a "stop" instruction and react. The 3-second window gives them reaction time before the nav instruction could be confused with the danger alert.

7. The 0.10 FL Prior Strength This constant is the result of deliberate tuning. With `_kFlPriorStrength = 0.10`, a class with global weight 0.44 (a well-detected class across devices) gets a +0.044 boost. This is enough to lift a near-threshold detection (0.306 → 0.35) but not large enough to cause false positives from completely absent objects.

8. Offline-First by Default The FL weights load from local cache first, then refresh asynchronously. Settings come from SharedPreferences first, then sync from Firestore. The model is pre-bundled in the APK. The app works fully offline from day one. Firebase is a synchronization layer, not a dependency.

Summary Table — All Features at a Glance

Feature	Input	Processing	Output	Key Technology
Obstacle Detection	Live camera (YUV420)	TFLite YOLO/SSD → DecisionEngine	TTS alert	tflite_flutter, camera
GPS Navigation	Voice/text destination	Nominatim + OSRM + DecisionEngine	TTS turn-by-turn	geolocator, http
OCR (English)	Photo	ML Kit TextRecognizer	TTS reading	google_mlkit
OCR (Arabic)	Photo	Gemini Vision API	TTS reading	http + Gemini
Scene Description	Photo	Gemini Vision API	TTS description	http + Gemini
Emergency SMS	Button/voice	url_launcher	SMS app opened	url_launcher

Feature	Input	Processing	Output	Key Technology
Emergency Call	Button/voice	url_launcher	Dialer opened	url_launcher
Voice Commands	Microphone	speech_to_text → command map	Screen action	speech_to_text
Settings Sync	Any setting change	SharedPreferences + Firestore	Persistent + cloud	cloud_firestore
Federated Learning	Camera frames	80-dim embeddings → Firebase	Better detection	firebase_storage
Language Switch	Settings tap	LocaleNotifier + TtsHelper	UI rebuilds RTL	flutter_localizations

This document was generated through deep static analysis of the complete source tree. All code references, algorithms, and architectural patterns described here are directly derived from the implementation files.